

LFA - Tema 3

Maria Preda

Precizări

Tema se realizează individual și fără utilizarea modelelor de tip LLM.

Deadline: 26.05.2026, ora 23:59.

Nerespectarea formatului specificat pentru input și output va fi notată cu 0 puncte.

1. Simulator de Automate Push-Down (PDA)

Implementați un simulator pentru un automat cu stivă (PDA).

Cerințe:

1. Programul trebuie să citească definiția unui PDA și un șir de intrare.
2. Trebuie să simuleze execuția automatului, inclusiv operațiile pe stivă: push și pop.
3. Trebuie tratat nedeterminismul prin explorarea mai multor configurații posibile.
4. Acceptarea se face conform definiției: prin stare finală și/sau stivă goală, în funcție de specificații.

Fișier Input (fiecare linie este o linie din input):

- Mulțimea stărilor
- Alfabetul de intrare
- Numărul de tranziții
- Pe câte o linie, fiecare tranziție, sub forma:

stare_curentă simbol_citit simbol_vârș_stivă stare_următoare șir_de_pus_pe_stivă

unde:

- *simbol_citit* poate fi λ dacă tranziția nu consumă niciun simbol din input;

- `simbol_vârf_stivă` este simbolul care trebuie eliminat de pe stivă;
- `șir_de_pus_pe_stivă` poate fi λ dacă nu se pune nimic pe stivă.
- Starea inițială
- Simbolul inițial al stivei
- Mulțimea stărilor finale
- Modul de acceptare: `stare_finală`, `stivă_goală` sau ambele
- Cuvântul de verificat

Fișier Output: Acceptat sau Respins.

2. Transformarea unei gramatici în Formă Normală Chomsky (FNC)

Implementați un program care transformă o gramatică independentă de context (CFG) dată într-o gramatică echivalentă în Formă Normală Chomsky.

Cerințe:

1. Eliminarea λ -producțiilor (producții vide).
2. Eliminarea producțiilor unitare (de forma $A \rightarrow B$).
3. Eliminarea simbolurilor neproductive și inaccesibile.
4. Transformarea producțiilor rămase astfel încât să fie de forma $A \rightarrow BC$ sau $A \rightarrow a$.

Fișier Input:

- Simbolurile neterminale, terminale și producțiile (identic cu formatul de la Exercițiul 1).
- Simbolul de start.

Fișier Output:

- Gramatica rezultată în format: `Neterminal -> Șir_Simboluri`.
- Fiecare producție pe o linie nouă.

Exemplu de fișier:

```
S A
a b
3
S aA
A bA
A b
S
3
```

Observație: Pentru toate cerințele la care aveți ca input o gramatică, acesta va fi formatul. În cazul în care aveți de scris într-un fișier outputul sub forma unei gramatici, îl veți respecta tot pe acesta din input.

3. Algoritmul CYK (Cocke-Younger-Kasami)

Implementați algoritmul CYK pentru a verifica dacă un cuvânt dat aparține limbajului generat de o gramatică în Formă Normală Chomsky.

Cerințe:

1. Programul primește o gramatică deja transformată în FNC.
2. Utilizează o tabelă de programare dinamică pentru a verifica apartenența cuvântului.
3. Afișează tabela rezultată pentru vizualizarea procesului de derivare.

Fișier Input:

- Gramatica în FNC (formatul standard).
- Cuvântul w care trebuie verificat.

Fișier Output:

- Mesajul DA sau NU.
- Tabela CYK sub formă de matrice.

Bonus — Parser LR(1)

Realizați un parser sintactic LR(1) pentru un mini-limbaj care suportă: atribuiri, expresii aritmetice, instrucțiuni condiționale `if`, bucle `while`.

Cerințe:

1. Modelați o astfel de gramatică.
2. Citiți definiția gramaticii dintr-un fișier de intrare.
3. Construiți automat colecția de itemi LR(1).
4. Generați tabelele `ACTION` și `GOTO`.
5. Implementați algoritmul de parsare LR(1).
6. Afișați pașii parsării și indicați dacă șirul este acceptat sau respins.

Fișier Input:

- Gramatica limbajului;
- Șirul de tokeni care trebuie analizat sintactic.

Fișier Output:

- Colecția de itemi LR(1);
- Tabelele ACTION și GOTO;
- Succesiunea pașilor efectuați de parser;
- Mesaj final:
 - ACCEPTAT — dacă șirul respectă gramatica;
 - EROARE SINTACTICĂ — în caz contrar.